

AASD 4016

Full Stack Data Science Systems

Applied AI Solutions Developer Program



Module 2

Flask

Vejeý Gandýer

Agenda

Flask

Requests

Response

Routes

Examples

Flask

Python Web Framework



Flask

Installing Flask

```
pip install Flask
```

Hello Flask Application (hello.py)

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def hello_world():
```

```
    return "<p>Hello, World!</p>"
```

Flask

Running a Flask Application

```
export FLASK_APP=hello  
flask run
```

Set Flask Environment

```
export FLASK_ENV=development
```

Project Structure

```
/home/user/Projects/flask-tutorial
├── flask/
│   ├── __init__.py
│   ├── db.py
│   ├── schema.sql
│   ├── auth.py
│   ├── blog.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── auth/
│   │   │   ├── login.html
│   │   │   └── register.html
│   │   └── blog/
│   │       ├── create.html
│   │       ├── index.html
│   │       └── update.html
│   └── static/
│       └── style.css
├── tests/
│   ├── conftest.py
│   ├── data.sql
│   ├── test_factory.py
│   ├── test_db.py
│   ├── test_auth.py
│   └── test_blog.py
├── venv/
├── setup.py
└── MANIFEST.in
```

Flask project structure

Project Structure

`app.py`

`helper(s).py` [Optional]

Iris Classifier

Notebook to Script

Script to Flask

Load model and keep it up for inference

Infer from the model



HTTP Request Methods

GET
POST
PUT
HEAD
DELETE

GET

- GET is used to request data from a resource
- Query String is sent in the URL of a GET request
 - Developer Tools
 - `/test_endpoint?name=subash&age=41`

POST

- POST is used to send data to a server to create / update a resource
- Data sent to the server is stored in the request body of the HTTP request
 - Developer Tools
 - POST /test_endpoint HTTP/1.1
Host: iamsubash.com
name=subash&age=41

PUT

- PUT is used to send data to a server to create / update a resource
- Calling PUT request multiple times produce the same result
- Calling POST request multiple times produce multiple resources

HEAD

- HEAD is identical to GET, but without the response body
- HEAD requests are useful for checking what a GET request will return before actually making an actual GET request
 - Before downloading a very large file

DELETE

- DELETE deletes the specified resource

GET vs POST

	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted (the browser should alert the user that the data are about to be re-submitted)
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
Encoding type	application/x-www-form-urlencoded	application/x-www-form-urlencoded or multipart/form-data. Use multipart encoding for binary data
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters)	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions. Binary data is also allowed
Security	GET is less secure compared to POST because data sent is part of the URL Never use GET when sending passwords or other sensitive information!	POST is a little safer than GET because the parameters are not stored in browser history or in web server logs
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

Flask HTTP Methods

HTTP Methods

```
from flask import request
```

```
@app.route('/login', methods=['GET', 'POST'])  
def login():  
    if request.method == 'POST':  
        return do_the_login()  
    else:  
        return show_the_login_form()
```

Rendering Templates

```
from flask import render_template

@app.route('/hello/')
@app.route('/hello/<name>')
def hello(name=None):
    return render_template('hello.html', name=name)
```

File uploads

```
from flask import request
```

```
@app.route('/upload', methods=['GET', 'POST'])
```

```
def upload_file():
```

```
    if request.method == 'POST':
```

```
        f = request.files['the_file']
```

```
        f.save('/var/www/uploads/uploaded_file.txt')
```

```
    ...
```

A horizontal bar at the bottom of the slide composed of several colored squares: orange, yellow, light purple, dark purple, light blue, dark blue, yellow, light green, and dark green.

Secure File uploads

```
from werkzeug.utils import secure_filename

@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        f = request.files['the_file']
        f.save(f'/var/www/uploads/
{secure_filename(f.filename)}')
    ...
```

Further Reading

Flask Tutorial

<https://flask.palletsprojects.com/en/2.0.x/tutorial/>